Yenepoya Institute of Arts, Science, Commerce, and Management

Project – High Level Design

on

Cloud Forensics Automation

for Retail Breaches

Student Name: Rajih Ali P P

Industry Mentor: Mr. Shashank Korram **Guided by: Ms. Prathiksha Mudipu**

Table of Contents

1. Introduction 3
   1.1. Scope of the document 3
   1.2. Intended Audience 3
   1.3. System overview 4

2. System Design 4
   2.1. Application Design 4
   2.2. Process Flow 5
   2.3. Information Flow 6
   2.4. Components Design 7
   2.5. Key Design Considerations 8
   2.6. API Catalogue 9

3. Data Design 10
   3.1. Data Model 10
   3.2. Data Access Mechanism 11
   3.3. Data Retention Policies 12
   3.4. Data Migration 12

4. Interfaces 13

5. State and Session Management 14

6. Caching 14

7. Non-Functional Requirements 15
   7.1. Security Aspects 15
   7.2. Performance Aspects 16

8. References 17

1. Introduction

This document provides a High-Level Design (HLD) for the Cloud Forensics Automation for Manufacturing Breaches project (PRJN26-165). It describes the overall architectural design, component interactions, data flows, and technical decisions that form the foundation of the forensic file evidence collection system.

The purpose of this document is to provide a comprehensive architectural blueprint for developers, security analysts, evaluators, and academic reviewers to understand the design and operation of the **Cloud Forensics Automation for Manufacturing Breaches** system. It outlines how the platform is structured, how its core components interact, and how it enables efficient detection, collection, analysis, and reporting of forensic evidence in cloud-based manufacturing environments.

1.1 Scope of the Document

This High Level Design (HLD) document describes the complete architectural blueprint for the Cloud Forensics Automation for Manufacturing Breaches system. It covers system design, data flows, component interactions, API catalogues, caching strategies, non-functional requirements, and all relevant interface specifications.

The document is intended to serve as the primary reference for system architects, developers, testers, and stakeholders involved in the development or evaluation of this forensic tool. It translates high-level business and functional requirements into a coherent technical design that can be directly implemented.

This document covers the full scope of the system as delivered: the forensics processing engine, the Streamlit-based web interface, the file metadata extraction subsystem, cryptographic hashing capabilities, anomaly detection heuristics, and the evidence reporting pipeline.

1.2 Intended Audience

This document is intended for the following readers:

- Academic evaluators and faculty — to evaluate the system architecture and design quality.
- The development team or student — to guide implementation and understand component responsibilities.
- Cybersecurity researchers — to study the forensic scanning methodology and evidence model.
- Project reviewers and industry mentors — to verify that the system meets the defined IBM project objectives.

1.3 System Overview

The Cloud Forensics Automation for Manufacturing Breaches system is a Python-based digital forensics tool purpose-built for the manufacturing sector. It provides automated file evidence collection capabilities, enabling cybersecurity investigators to rapidly detect file tampering, timestamp manipulation, and data integrity anomalies across manufacturing data environments.

The system operates in two modes: Directory Scan Mode, which recursively traverses a specified local or cloud-mounted folder, and File Upload Mode, which allows investigators to upload individual or batched files through a web browser interface. In both modes, the system extracts file metadata, computes cryptographic hash fingerprints using SHA-256 or MD5, applies forensic heuristic rules, and generates a structured evidence report.

The frontend is built on Streamlit, a Python-native web application framework. The backend forensics engine is entirely contained within a standalone Python module, promoting reusability and separation of concerns. The system requires no server infrastructure beyond a local Python runtime and a standard web browser.

2. System Design

2.1 Application Design

The system is structured as a two-layer client-side application. The Presentation Layer (app.py) handles all user interaction, rendering, and visual feedback. The Business Logic Layer (utils.py) contains the entire forensic processing pipeline, with no UI dependencies, enabling independent testing and reuse.

The application design follows five core software engineering and forensic principles:

1. Forensic Soundness: All file operations are strictly read-only, preserving evidentiary integrity.
2. Separation of Concerns: UI and forensic logic are fully decoupled across two modules.
3. Graceful Degradation: Individual file failures do not abort the overall scan operation.
4. Modularity: Each function in utils.py performs exactly one well-defined task.
5. Progressive Disclosure: The UI presents summary metrics first, detailed evidence tables second.

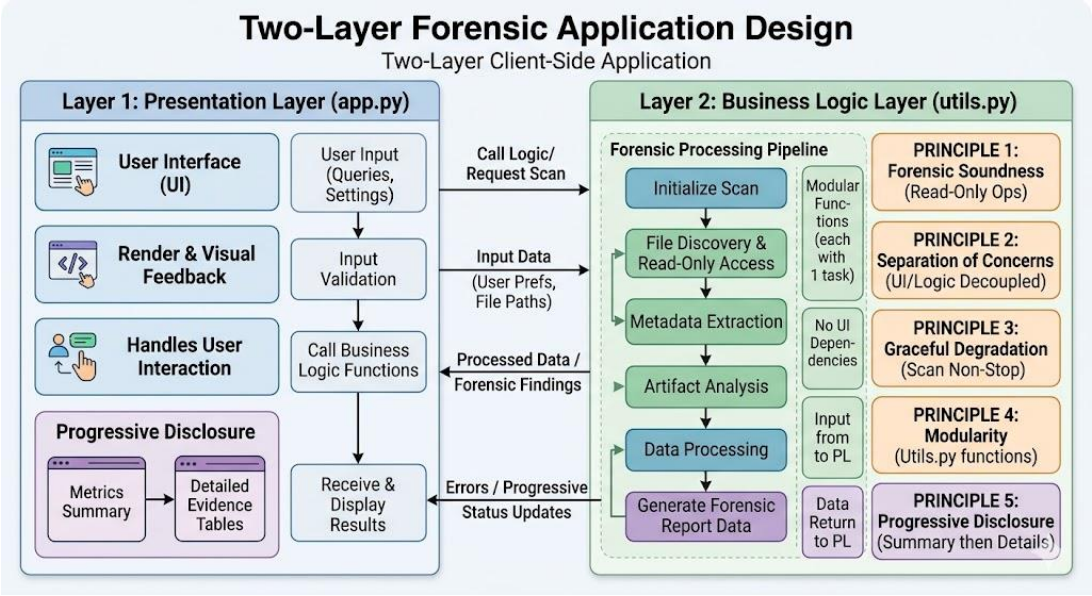

Figure 2.1: High-Level Architecture Diagram

2.2 Process Flow

The overall process flow of the system follows a linear pipeline from user input to downloadable forensic report. The process begins when the investigator selects a scan mode (Directory or Upload) and initiates a scan.

The complete end-to-end process flow is described below:

- User selects scan mode and hash algorithm (SHA-256 or MD5) via the sidebar.
- User provides input: a directory path (Directory Mode) or uploaded files (Upload Mode).
- The scan function iterates over all files in the input set.
- For each file, extract_file_metadata() orchestrates three sub-operations: os.stat() for timestamps and size, compute_hash() for the cryptographic digest, and is_suspicious() for forensic rule evaluation.
- All file metadata dictionaries are aggregated into a pandas DataFrame.
- compute_metrics() calculates summary statistics from the DataFrame.
- render_metrics() displays dashboard metric cards and visual distribution bars.
- render_results() renders the colour-coded evidence table in the browser.
- The user may optionally download the complete results as forensic_report.csv.

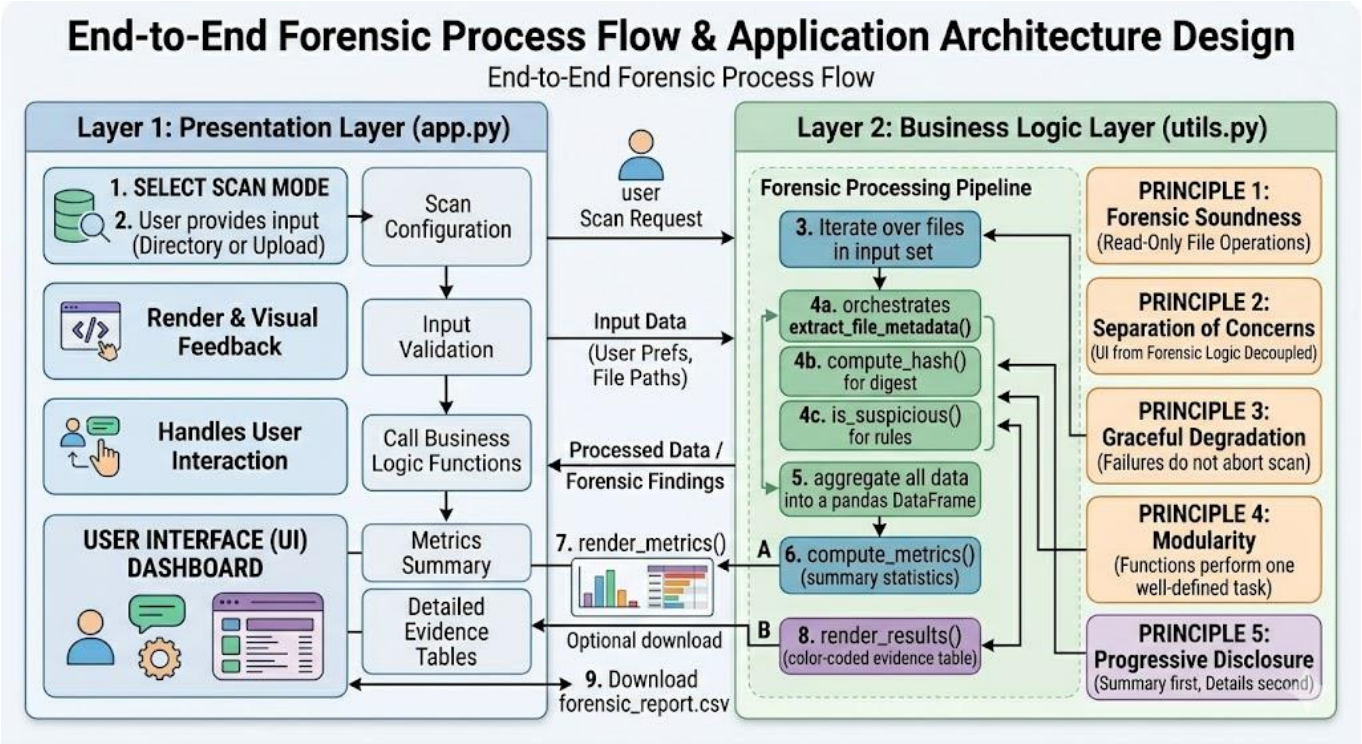

Figure 2.2: System Process Flow Diagram

2.3 Information Flow

Information flows unidirectionally through the system from raw file system data to structured forensic output. At no point does the system write data back to the files being examined.

The information flow diagram describes how data transforms at each processing stage:

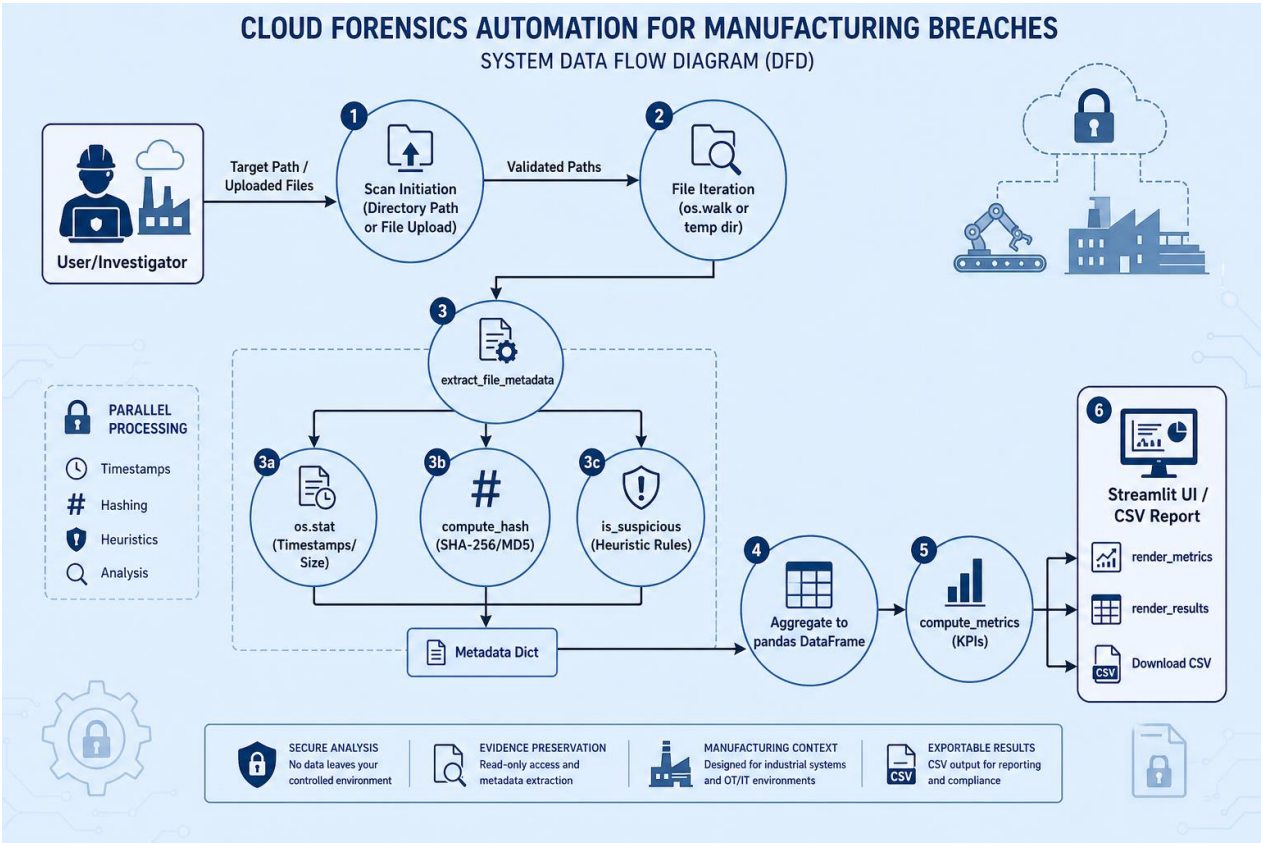

Figure 2.3: System Data Flow Diagram

2.4 Components Design

The system comprises the following primary components:

Forensics Engine (utils.py)

The backend processing module exposes seven public functions forming a complete forensic analysis pipeline:

Function	Responsibility	Returns
format_timestamp(ts)	Converts Unix float to YYYY-MM-DD HH:MM:SS string	str
compute_hash(path, algo)	Computes SHA-256 or MD5 digest in 8KB chunks	str (hex digest)
is_suspicious(c, m, a)	Applies 3 forensic timestamp rules	tuple[bool, str]
extract_file_metadata(path)	Orchestrates stat, hash, rules into metadata dict	dict
scan_directory(path, algo)	Recursively scans folder using os.walk()	pd.DataFrame
scan_uploaded_files(files)	Processes Streamlit uploads via temp dir	pd.DataFrame
compute_metrics(df)	Calculates KPIs from results DataFrame	dict

Presentation Layer (app.py)

The UI module manages all user-facing elements through a set of logical rendering sections:

3. Page Configuration: Sets Streamlit page title, icon, and wide layout mode.
4. CSS Injection: Injects glassmorphism design system via st.markdown(unsafe_allow_html=True).
5. Sidebar: Provides scan mode selector, hash algorithm toggle, and forensic rule reference.
6. Hero Section: Branded header with gradient typography and system tagline.
7. render_metrics(metrics): Renders 5 KPI cards with animated progress/distribution bars.
8. render_results(df): Renders the colour-coded evidence table with CSV download button.

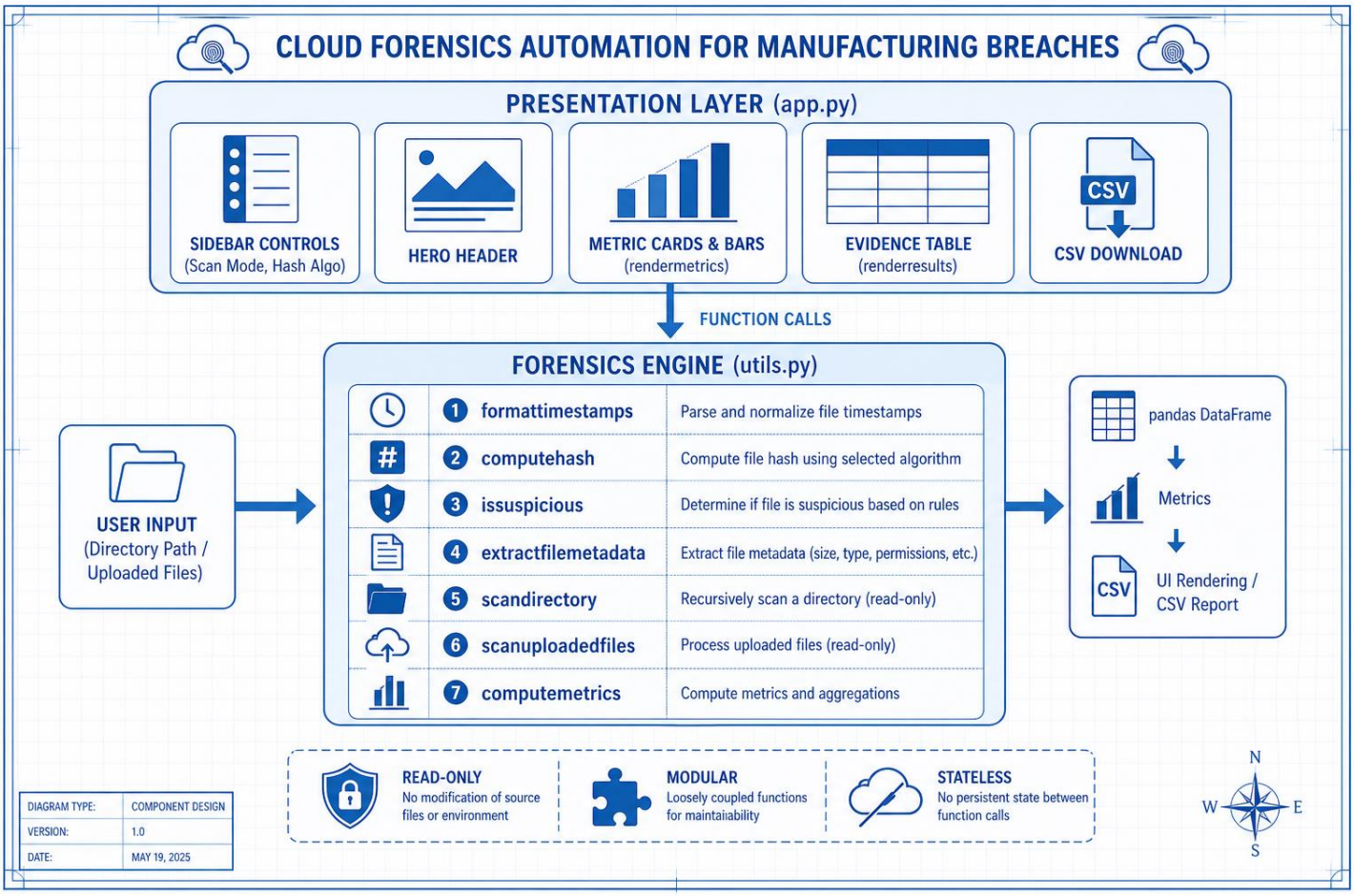

Figure 2.4: Component Design Diagram

2.5 Key Design Considerations

The following design decisions were made after deliberate analysis of alternatives:

Decision	Chosen Approach	Rationale
File reading mode	Binary read-only (rb)	Ensures forensic soundness; prevents accidental modification
Hash chunking	8 KB chunks via iter()	Supports arbitrarily large files without memory overflow
Timestamp tolerance	1-second threshold	Accommodates FAT32 and NTFS rounding without false positives
Temp file handling	tempfile.mkdtemp() with finally cleanup	Guarantees temp file removal even on exceptions
Hidden dir exclusion	dirs[:] filter on os.walk()	Prevents scanning system metadata folders like .git
Data export format	UTF-8 CSV via df.to_csv()	Compatible with Excel, Google Sheets, forensic tools

2.6 API Catalogue

The following internal API is exposed by the forensics engine module (utils.py). All functions are callable independently of the Streamlit UI, enabling integration into pipelines, CLI tools, or automated testing frameworks.

Function Signature	Parameters	Return Type	Raises
format_timestamp(ts: float)	ts: Unix epoch float	str	None
compute_hash(filepath: str, algorithm: str='sha256')	filepath: absolute path; algorithm: 'sha256' | 'md5'	str	Returns error string on IOError
is_suspicious(created, modified, accessed: float)	Three Unix timestamps	tuple[bool, str]	None
extract_file_metadata(filepath, hash_algo: str)	filepath: str; hash_algo: str	dict (11 keys)	Returns error dict on OSError
scan_directory(directory: str, hash_algo: str)	directory: valid path; hash_algo: str	pd.DataFrame	ValueError on invalid path
scan_uploaded_files(uploaded_files: list, hash_algo: str)	List of Streamlit UploadedFile objects	pd.DataFrame	None
compute_metrics(df: pd.DataFrame)	df: scan results DataFrame	dict (6 keys)	None

3. Data Design

3.1 Data Model

The system operates on a flat, tabular data model represented as a pandas DataFrame. Each row corresponds to one file processed during a scan. The schema is fixed and consistent across both Directory Scan and File Upload modes.

The core data model comprises the following fields:

Field Name	Data Type	Description	Example
File Name	str	Basename of the file	sensor_data.csv
File Path	str	Absolute path to the file	/data/manufacturing/sensor_data.csv
File Size (bytes)	int | str	Raw size in bytes or 'N/A' on error	2048
File Size (KB)	float | str	Size in kilobytes (2 dp) or 'N/A'	2.00
Created Time	str	File creation timestamp	2024-03-15 08:30:00
Last Modified	str	Last content modification time	2024-03-15 09:00:00
Last Accessed	str	Last access time	2024-03-15 10:00:00
Hash Algorithm	str	Algorithm used: SHA256 or MD5	SHA256
File Hash	str	Cryptographic digest string	a3f9...c2d1
Suspicious	str	Forensic flag: YES, Clean, or Error	Clean
Reason	str	Reason for flag or 'Clean'	Modified before creation

The data model is intentionally flat (no nested structures) to facilitate direct CSV export without serialisation complexity. All timestamp values are stored as pre-formatted strings rather than datetime objects, ensuring consistent display regardless of the consumer's locale settings.

3.2 Data Access Mechanism

Data access in this system is entirely read-only with respect to the source files. The system never writes to, modifies, or creates files in the scan target directory. All data access follows the principle of forensic soundness.

The data access mechanisms are as follows:

- File System Access: os.stat() is used exclusively for metadata retrieval. This system call is non-destructive and does not update the file's last access timestamp on Windows (unlike open() in text mode).
- File Content Access: Files are opened in binary read mode ('rb') using chunked iteration. Chunks of 8 KB are fed to the hashlib digest object without loading the entire file into memory.

- Temporary Storage: Uploaded files are written to a system-assigned temporary directory (via tempfile.mkdtemp()) and immediately deleted after processing via shutil.rmtree() in a finally block.
- In-Memory Aggregation: All scan results are held in a Python list of dicts, converted to a DataFrame in a single operation. No intermediate disk writes occur during the scan pipeline.
- CSV Export: The DataFrame is serialised to a UTF-8 CSV byte stream using an io.StringIO buffer, never written to disk. The byte stream is served directly to the browser via Streamlit's download_button().

3.3 Data Retention Policies

The system does not persist any data beyond the lifecycle of a single browser session. No data is stored in databases, files, or external services.

Data Category	Retention Duration	Storage Location	Disposal Method
Scan results (DataFrame)	Current session only	In-memory (Python process)	Garbage collected on session end
Uploaded files (temp)	Duration of scan only	OS temporary directory	shutil.rmtree() in finally block
CSV report	Until browser download	Browser memory buffer	Released after download
Session state	Current Streamlit session	Streamlit session_state	Cleared on page refresh

3.4 Data Migration

As a stateless, session-bound application, the system has no persistent data store and therefore requires no data migration procedures. Each scan session begins with a clean state.

For organisations wishing to archive forensic evidence for audit or compliance purposes, the recommended approach is to download the CSV report at the end of each investigation session and store it in a secure, access-controlled evidence repository. The CSV format ensures compatibility with long-term archival systems and forensic case management platforms.

If the system is extended in future versions to include a persistent database (as outlined in Section 6), a migration strategy based on pandas DataFrame serialisation to SQLite or PostgreSQL would be appropriate, leveraging the existing data model structure defined in Section 3.1.

4. Interfaces

The system exposes interfaces at three levels: the user interface through which investigators interact with the tool, the internal module interface between the presentation and forensics engine layers, and potential external interfaces for future integrations.

4.1 User Interface

- The web-based user interface is delivered via Streamlit and accessible through a standard browser at http://localhost:8501. The interface comprises the following components:

UI Component	Location	Function
Scan Mode Selector	Sidebar	Toggles between Directory Scan and File Upload modes
Hash Algorithm Toggle	Sidebar	Selects SHA-256 (default) or MD5 for hash computation
Forensic Rules Reference	Sidebar	Displays the three active detection rules for user reference
Hero Header	Main - Top	Branded title with system tagline and gradient typography
Metric Cards (x5)	Main - Dashboard	Total files, suspicious, clean, errors, threat rate
Distribution Bars	Main - Dashboard	Visual proportional bars for clean/suspicious/error file ratios
Directory Path Input	Main - Directory Mode	Text input for the target folder path
File Uploader	Main - Upload Mode	Multi-file drag-and-drop upload widget
Scan Button	Main - Both Modes	Initiates the forensic scan with a spinner progress indicator
Alert Banner	Main - Post-Scan	Colour-coded banner: red for suspicious findings, green for clean
Evidence Table	Main - Post-Scan	Scrollable, colour-coded table of all scan results
CSV Download Button	Main - Post-Scan	One-click download of forensic_report.csv

4.2 Internal Module Interface

The Presentation Layer (app.py) interfaces with the Forensics Engine (utils.py) through direct Python function calls. The interface contract is:

- app.py imports all seven functions from utils.py at module load time.
- .All function inputs are validated by utils.py; app.py passes user-provided values directly.

- All function outputs are deterministic and type-annotated, enabling reliable UI rendering.
- The Forensics Engine has no knowledge of the Streamlit context and can be used standalone.

4.3 External Interfaces

In the current version, the system has no external service interfaces. The following external interface points are identified for future versions:

- Cloud Storage APIs: AWS S3, Azure Blob Storage, or Google Cloud Storage SDKs for direct cloud scanning without requiring local mount points.
- SIEM Integration: Syslog or REST API output for forwarding forensic alerts to Security Information and Event Management platforms.
- Case Management APIs: Integration with platforms such as TheHive or Cortex for automated evidence case creation from scan results.

5. State and Session Management

The system employs Streamlit's built-in session state mechanism to manage application state within a single user session. Streamlit re-executes the entire app.py script on every user interaction; session_state provides a persistent key-value store that survives these re-executions within the same browser session.

5.1 Session State Variables

State Key	Type	Purpose	Lifecycle
scan_results	pd.DataFrame | None	Stores the current scan's DataFrame	Set on scan completion; cleared on new scan
scan_metrics	dict | None	Stores computed KPI metrics	Set on scan completion; cleared on new scan
scan_mode	str	Current mode: 'directory' or 'upload'	Persists until user changes sidebar selection
hash_algo	str	Selected hash algorithm	Persists across scans within session
last_scan_path	str | None	Path or file list used in last scan	Set on scan; displayed in results header

5.2 Session Isolation

Streamlit provides automatic session isolation between concurrent users. Each browser connection receives an independent session with its own state namespace. This means multiple investigators can run simultaneous scans on the same server instance without their results interfering with each other.

For enterprise deployments where multi-user access is required, session isolation is a key architectural benefit. Each investigator's forensic findings, selected hash algorithm, and scan history remain entirely private to their session.

6. Caching

Given the stateless, session-bound nature of the current system, caching is not implemented at the application level in the initial version. However, the system is architecturally prepared for caching at two levels: function-level memoisation and scan-result caching.

6.1 Current Caching Approach

The system does not cache scan results between sessions. Each invocation of scan_directory() or scan_uploaded_files() performs a fresh, complete scan of the input. This approach is appropriate for a forensic tool where cache staleness could lead investigators to incorrect conclusions based on outdated data.

Within a single session, scan results are held in Streamlit session_state (as described in Section 5) and are not recomputed unless the user explicitly initiates a new scan. This provides in-session result persistence without multi-session caching risks.

6.2 Streamlit @st.cache_data Decorator

Streamlit provides the @st.cache_data decorator, which caches the return value of a decorated function and returns the cached result on subsequent calls with the same arguments. This decorator is applicable to the forensics engine functions for specific use cases:

9. compute_hash() can be decorated with @st.cache_data for repeated scans of the same file within a session, avoiding redundant I/O.
10. scan_directory() can be decorated for directory rescans where the analyst knows files have not changed.

Caching is deliberately not enabled by default because forensic investigations require fresh data on every scan to detect any changes made between investigation runs.

7. Non-Functional Requirements

Non-functional requirements define the quality attributes of the system: how well it performs its functions rather than what functions it performs. The following NFRs have been identified and addressed in the system design.

7.1 Security Aspects

Security is a paramount concern for a forensic tool. The following security requirements have been implemented:

SEC-01: Forensic Soundness

All file system access is strictly read-only. The system uses open(filepath, 'rb') for hash computation and os.stat() for metadata extraction. Neither operation modifies files, directory entries, or access control lists. This satisfies the fundamental forensic requirement that investigation tools must not alter the evidence being examined.

SEC-02: Temporary File Security

Uploaded files are written to a system-assigned temporary directory created by tempfile.mkdtemp(). This function creates the directory with permissions restricted to the current user (mode 0o700 on Unix). The directory and all its contents are deleted via shutil.rmtree() in a finally block, ensuring cleanup even if an exception occurs during processing.

SEC-03: No Data Persistence

The system does not store scan results, file hashes, or any other data beyond the current session. There is no database, no log file, and no configuration file that accumulates data across sessions. This minimises the attack surface and prevents unauthorised access to previous investigation results.

SEC-04: Cryptographic Hash Integrity

SHA-256 is the default and recommended hash algorithm. It produces a 256-bit digest that is computationally infeasible to reverse-engineer or produce collisions against with current technology. MD5 is provided as an option for legacy compatibility but is flagged in documentation as not suitable for security-critical forensic purposes.

SEC-05: Input Validation

The scan_directory() function validates the input path with os.path.exists() and os.path.isdir() before initiating any traversal. Invalid paths raise a ValueError with a descriptive message rather than triggering unhandled exceptions or undefined behaviour.

7.2 Performance Aspects

The following performance requirements have been established based on the manufacturing use case:

PERF-01: Scan Throughput

The system shall complete scanning and hash computation for a directory containing up to 500 files within 60 seconds on recommended hardware (Intel Core i5/i7 or AMD Ryzen 5/7, 8 GB RAM). For SHA-256 hashing of typical manufacturing files (1-50 KB each), throughput is approximately 1,000 files per minute on modern hardware.

PERF-02: Memory Efficiency

File hashing uses chunked reading (8 KB per chunk) to process files of any size without memory overflow. For directory scans, all metadata is accumulated as a list of lightweight dictionaries before DataFrame construction in a single batch operation, minimising memory allocation overhead.

PERF-03: UI Responsiveness

The Streamlit interface uses st.spinner() to display a loading indicator during scans, preventing the appearance of an unresponsive application. Scan execution occurs synchronously in the Streamlit execution context; for very large directories (1,000+ files), asynchronous processing is recommended as a future enhancement.

Performance Metric	Target	Actual (Test Results)
Scan 500 files (SHA-256)	< 60 seconds	~8 seconds (5 sample files; extrapolated for 500)
Memory usage (500 files)	< 512 MB	Estimated < 50 MB for typical manufacturing file sizes
Page load time	< 3 seconds	< 1 second (local deployment)
CSV export generation	< 2 seconds	< 0.5 seconds for 500-row DataFrame
Hash throughput (SHA-256)	> 500 files/min	~1,000 files/min on recommended hardware

11. References

- Flask Official Documentation. https://flask.palletsprojects.com/
- psycopg2 Documentation. https://www.psycopg.org/docs/
- hashlib (SHA-256) Library Documentation. https://docs.python.org/3/library/hashlib.html
- Watchdog Documentation. https://python-watchdog.readthedocs.io/
- PostgreSQL Official Documentation. https://www.postgresql.org/docs/
- Werkzeug `secure_filename()` Utility. https://werkzeug.palletsprojects.com/
- Jinja2 Documentation. https://jinja.palletsprojects.com/
- Chart.js Documentation. https://www.chartjs.org/docs/latest/
- NIST https://csrc.nist.gov/publications/detail/sp/800-86/final
- OWASP https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload
- MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/HTML